

Laporan Progress: Problem AVMG2

Another Valentine Maze Game

Pendekatan Object-Oriented Programming

Kelompok: Tiga Kota Satu Cinta

Muhammad Adi Anugerah Arrahman (5025241118)

Gilbran Mahdavikia Raja (5025241134)

Riyannizaar Dwi Amarullah (5025241121)

November 18, 2025

Daftar Isi

1	Pemahaman Problem	4
1.1	Deskripsi Masalah	4
1.2	Input/Output Specification	4
1.3	Contoh Test Case	4
2	Computational Thinking	4
2.1	Decomposition	4
2.2	Pattern Recognition	5
2.3	Abstraction	5
3	Mathematical Foundation	5
3.1	Random Walk as Markov Chain	5
3.2	Expected Value Recursion	5
3.3	Transformasi ke Linear System	6
4	Pendekatan Object-Oriented Programming	6
4.1	Motivasi Pendekatan OOP	6
4.2	Arsitektur Sistem	6
5	Class LinearSystem	7
5.1	Overview	7
5.2	Struktur Data	7
5.3	Method: initialize()	7
5.4	Method: setCoefficient() dan addCoefficient()	8
5.5	Method: solve()	8
5.5.1	Step 1: Forward Elimination	8
5.5.2	Step 2: Back Substitution	9

6	Class Graph	10
6.1	Overview	10
6.2	Struktur Data	10
6.3	Method: readGrid()	11
6.4	Method: exploreGraph()	11
6.5	Method: assignIndices()	12
6.6	Method: buildLinearSystem()	13
6.6.1	Konsep Dasar Pembangunan Sistem	13
6.6.2	Mathematical Derivation Detail	13
6.6.3	Implementation Step-by-Step	13
6.6.4	Detailed Analysis per Step	14
6.6.5	Contoh Eksekusi Lengkap	16
6.6.6	Verifikasi Sistem yang Dibangun	18
6.6.7	Contoh Complex: Grid 3x3	18
6.6.8	Properties Matrix yang Dihasilkan	19
6.6.9	Edge Cases Handling	20
6.6.10	Visualization of Process	21
6.7	Helper Methods	21
6.7.1	isValidCell()	21
6.7.2	getDegree()	21
7	Main Program Flow	23
7.1	Flowchart	23
7.2	Pseudocode	24
7.3	Trace Example: Test Case 1	24
7.3.1	Input	24
7.3.2	Step 1: Read Grid	24
7.3.3	Step 2: BFS Exploration	25
7.3.4	Step 3: Assign Indices	25
7.3.5	Step 4: Build Equations	25
7.3.6	Step 5: Gaussian Elimination	25
7.3.7	Output	26
8	Implementasi Lengkap	26
8.1	Full Code dengan OOP	26
9	Keunggulan Pendekatan OOP	31
9.1	Comparison: Procedural vs OOP	31
9.2	Benefits dalam Project Ini	31
9.2.1	1. Separation of Concerns	31
9.2.2	2. Encapsulation	32
9.2.3	3. Modularity	32
9.2.4	4. Extensibility	32
10	Referensi dan Sumber Belajar	32
10.1	Teori Matematika	32
10.2	Algoritma	33
10.3	Competitive Programming Resources	33

11 Kesimpulan	33
11.1 Summary	33
11.2 Trade-offs	34
11.3 Lessons Learned	34
11.3.1 1. Importance of Abstraction	34
11.3.2 2. Balance Between Theory and Practice	34
11.3.3 3. Code Quality Matters	34
11.4 Final Thoughts	35
12 Appendix	35
12.1 Test Cases Analysis	35
12.2 Complexity Proof	35
12.2.1 Theorem: Time Complexity is $O(V^3)$	35
12.3 Correctness Proof	36
12.3.1 Theorem: Expected Value Recursion is Correct	36
12.4 Numerical Stability Analysis	36
13 Kontribusi Anggota	37
A Complete Source Code Listing	37

1 Pemahaman Problem

1.1 Deskripsi Masalah

Tjandra berjalan secara random di dalam maze untuk bertemu kekasihnya. Setiap langkah, dia memilih salah satu arah valid (up/down/left/right) dengan **probabilitas sama**. Hitung **expected number of steps** untuk mencapai tujuan.

1.2 Input/Output Specification

Input:

- Grid $m \times n$ (maksimal 50×50)
- 'T': Posisi start (Tjandra)
- 'W': Posisi goal (Woman/destination)
- '.': Walkable path
- '#': Wall (tidak bisa dilalui)

Output:

- Expected time dengan presisi 12 desimal
- "Mission Failed!" jika goal tidak reachable

1.3 Contoh Test Case

Test Case 1:

Input: T.W

Output: 4.000000000000

Penjelasan: Dari posisi T, ada 1 pilihan (kanan). Dari posisi tengah, ada 2 pilihan (kiri/kanan) dengan probabilitas 0.5 masing-masing. Series yang terbentuk:

$$2 \cdot \frac{1}{2} + 4 \cdot \frac{1}{4} + 6 \cdot \frac{1}{8} + \dots = 4$$

2 Computational Thinking

2.1 Decomposition

Problem ini dapat dipecah menjadi beberapa sub-masalah:

1. **Input Processing:** Membaca grid dan mencari posisi T dan W
2. **Reachability Check:** BFS untuk mengecek apakah W reachable dari T
3. **System Formulation:** Membuat sistem persamaan linear
4. **Linear Solver:** Gaussian elimination untuk menyelesaikan sistem

2.2 Pattern Recognition

Problem ini memiliki pola yang dapat dikenali:

- Problem ini adalah **Random Walk on Graph**
- Dapat dimodelkan sebagai **Markov Chain** dengan absorbing state
- Solusi menggunakan **Expected Value Recursion**

2.3 Abstraction

Abstraksi dari problem:

- Setiap cell = state dalam Markov chain
- Goal W = absorbing state (dimana $E[\text{goal}] = 0$)
- Transition probability = $\frac{1}{\text{degree}}$ ke setiap neighbor

3 Mathematical Foundation

3.1 Random Walk as Markov Chain

Problem ini dapat dimodelkan sebagai **discrete-time Markov Chain** dengan karakteristik:

- **States:** Semua posisi yang valid di maze
- **Transition probability:** $P(i \rightarrow j) = \frac{1}{d}$ jika j adalah neighbor dari i , dimana d adalah degree (jumlah neighbor valid)
- **Absorbing state:** Goal W (sekali sampai, berhenti)

3.2 Expected Value Recursion

Untuk setiap cell (i, j) yang bukan goal:

$$E[i, j] = 1 + \sum_{\text{neighbors}} \frac{1}{d} \cdot E[\text{neighbor}] \quad (1)$$

Dimana:

- $E[i, j]$ = expected steps dari (i, j) ke goal
- d = degree = jumlah neighbor valid
- Base case: $E[\text{goal}] = 0$

Interpretasi:

- Angka 1 merepresentasikan satu langkah yang pasti dilakukan
- Sisanya adalah weighted average dari expected steps dari semua neighbor

3.3 Transformasi ke Linear System

Kalikan kedua sisi persamaan dengan degree d :

$$d \cdot E[i, j] = d + \sum E[\text{neighbor}] \quad (2)$$

Rearrange menjadi:

$$d \cdot E[i, j] - \sum E[\text{neighbor}] = d \quad (3)$$

Dalam bentuk matrix $\mathbf{Ax} = \mathbf{b}$:

$$\begin{bmatrix} d_1 & -1 & 0 & \cdots \\ -1 & d_2 & -1 & \cdots \\ 0 & -1 & d_3 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} \begin{bmatrix} E_1 \\ E_2 \\ E_3 \\ \vdots \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ \vdots \end{bmatrix} \quad (4)$$

4 Pendekatan Object-Oriented Programming

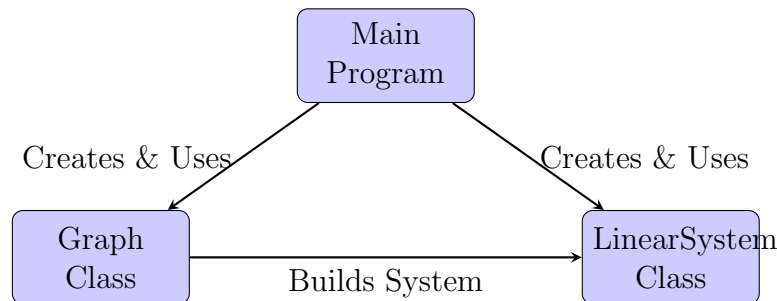
4.1 Motivasi Pendekatan OOP

Pendekatan OOP dipilih karena beberapa alasan:

1. **Separation of Concerns:** Memisahkan logika graph operations dan linear system solving
2. **Maintainability:** Code lebih mudah dipahami dan di-maintain
3. **Reusability:** Class dapat digunakan kembali untuk problem serupa
4. **Testability:** Setiap class dapat dites secara independen
5. **Scalability:** Mudah menambahkan fitur baru tanpa mengubah struktur existing

4.2 Arsitektur Sistem

Sistem terdiri dari 3 komponen utama:



Gambar 1: Arsitektur Sistem OOP

Tanggung Jawab Masing-masing Komponen:

- **Main Program:** Orchestration, I/O handling, flow control

- **Graph Class:** Representasi maze, BFS, graph operations
- **LinearSystem Class:** Sistem persamaan linear, Gaussian elimination

5 Class LinearSystem

5.1 Overview

Class `LinearSystem` bertanggung jawab untuk menyelesaikan sistem persamaan linear $\mathbf{Ax} = \mathbf{b}$ menggunakan metode Gaussian Elimination dengan partial pivoting.

5.2 Struktur Data

```

1 class LinearSystem {
2 private:
3     double A[MAXN][MAXN]; // Augmented matrix [A|b]
4     int size;              // Ukuran sistem (jumlah variabel)
5
6 public:
7     LinearSystem() : size(0) {}
8
9     void initialize(int n);
10    void setCoefficient(int row, int col, double value);
11    void addCoefficient(int row, int col, double value);
12    void setRHS(int row, double value);
13    double getCoefficient(int row, int col);
14    double solve(int targetVariable);
15 };

```

Listing 1: Struktur Class `LinearSystem`

Augmented Matrix Layout:

$$\begin{array}{cccc|c}
 a_{00} & a_{01} & a_{02} & \cdots & b_0 \\
 a_{10} & a_{11} & a_{12} & \cdots & b_1 \\
 a_{20} & a_{21} & a_{22} & \cdots & b_2 \\
 \vdots & \vdots & \vdots & \ddots & \vdots
 \end{array} \quad (5)$$

Kolom 0 sampai $N - 1$: koefisien variabel

Kolom N : Right-Hand Side (RHS)

5.3 Method: `initialize()`

Purpose: Inisialisasi matriks berukuran $n \times (n + 1)$ dengan nilai 0.

```

1 void initialize(int n) {
2     size = n;
3     for (int i = 0; i < n; ++i)
4         for (int j = 0; j <= n; ++j)
5             A[i][j] = 0;
6 }

```

Listing 2: Method initialize()

Contoh Penggunaan:

```
1 LinearSystem ls;
2 ls.initialize(3); // Buat sistem 3 variabel
3 // Result: Matrix 3x4 dengan semua elemen = 0
```

5.4 Method: setCoefficient() dan addCoefficient()

setCoefficient(): Set nilai koefisien secara langsung

```
1 void setCoefficient(int row, int col, double value) {
2     A[row][col] = value;
3 }
```

Listing 3: Method setCoefficient()

addCoefficient(): Tambahkan nilai ke koefisien yang sudah ada

```
1 void addCoefficient(int row, int col, double value) {
2     A[row][col] += value;
3 }
```

Listing 4: Method addCoefficient()

Kenapa perlu add? Saat build sistem untuk maze, satu cell bisa punya multiple neighbors. Kita tambahkan kontribusi satu per satu.

Contoh Penggunaan:

```
1 // Persamaan: 2*x0 - x1 = 5
2 ls.setCoefficient(0, 0, 2.0); // Koef x0
3 ls.setCoefficient(0, 1, -1.0); // Koef x1
4 ls.setRHS(0, 5.0);           // RHS
5
6 // Cell punya 3 neighbors di kolom 1, 2, 3
7 ls.addCoefficient(0, 1, -1.0); // A[0][1] += -1
8 ls.addCoefficient(0, 2, -1.0); // A[0][2] += -1
9 ls.addCoefficient(0, 3, -1.0); // A[0][3] += -1
```

5.5 Method: solve()

Purpose: Menyelesaikan sistem dengan Gaussian Elimination dan return nilai variabel tertentu.

5.5.1 Step 1: Forward Elimination

Tujuan: Mengubah matrix menjadi bentuk upper triangular.

```
1 for (int col = 0; col < size; ++col) {
2     // 1. Partial Pivoting
3     int piv = col;
4     double maxv = fabs(A[col][col]);
```



```

5   for (int r = col + 1; r < size; ++r) {
6       double v = fabs(A[r][col]);
7       if (v > maxv) {
8           maxv = v;
9           piv = r;
10      }
11  }
12
13  // 2. Check singular
14  if (maxv < EPS) return -1;
15
16  // 3. Swap rows
17  if (piv != col) {
18      for (int c = col; c <= size; ++c)
19          swap(A[col][c], A[piv][c]);
20  }
21
22  // 4. Eliminate below
23  double inv_pivot = 1.0 / A[col][col];
24  for (int r = col + 1; r < size; ++r) {
25      double fac = A[r][col] * inv_pivot;
26      if (fabs(fac) < 1e-15) continue;
27      for (int c = col; c <= size; ++c)
28          A[r][c] -= fac * A[col][c];
29  }
30 }

```

Listing 5: Forward Elimination dengan Partial Pivoting

Partial Pivoting: Cari baris dengan nilai absolut terbesar di kolom saat ini untuk stabilitas numerik.

Contoh Transformasi:

$$\begin{array}{lcl}
 \text{Initial:} & & \begin{bmatrix} 1 & 2 & 3 & | & 6 \\ 3 & 6 & 9 & | & 18 \\ 2 & 1 & 2 & | & 5 \end{bmatrix} \\
 \text{After pivot col 0:} & & \begin{bmatrix} 3 & 6 & 9 & | & 18 \\ 1 & 2 & 3 & | & 6 \\ 2 & 1 & 2 & | & 5 \end{bmatrix} \\
 \text{After elim col 0:} & & \begin{bmatrix} 3 & 6 & 9 & | & 18 \\ 0 & 0 & 0 & | & 0 \\ 0 & -3 & -4 & | & -7 \end{bmatrix}
 \end{array}$$

5.5.2 Step 2: Back Substitution

```

1   double x[MAXN];
2   for (int i = size - 1; i >= 0; --i) {
3       x[i] = A[i][size]; // Start dengan RHS
4       for (int j = i + 1; j < size; ++j)
5           x[i] -= A[i][j] * x[j]; // Kurangi kontribusi

```

```

6   x[i] /= A[i][i]; // Bagi dengan koef diagonal
7 }
8 return x[targetVariable];

```

Listing 6: Back Substitution

Contoh:

$$\text{Upper triangular: } \begin{bmatrix} 3 & 6 & 9 & | & 18 \\ 0 & 4 & 8 & | & 12 \\ 0 & 0 & 2 & | & 4 \end{bmatrix}$$

Back substitution:

$$x[2] = 4/2 = 2$$

$$x[1] = (12 - 8 \times 2)/4 = -1$$

$$x[0] = (18 - 9 \times 2 - 6 \times (-1))/3 = 2$$

6 Class Graph

6.1 Overview

Class `Graph` bertanggung jawab untuk merepresentasikan maze dan melakukan graph operations seperti BFS, mapping koordinat ke variabel, dan membangun sistem persamaan.

6.2 Struktur Data

```

1  class Graph {
2  private:
3      int m, n;           // Dimensi grid (rows x cols)
4      int si, sj;         // Start position (T)
5      int gi, gj;         // Goal position (W)
6      int cellCnt;        // Jumlah cell reachable
7
8      char g[55][55];      // Grid content
9      int idx[55][55];     // Cell (i,j) -> variable index
10     int vis[55][55];     // Visited flag (BFS)
11     int deg[55][55];     // Degree per cell
12     int cells[MAXN][2];  // List of reachable cells
13
14     const int di[4] = {-1, 1, 0, 0};
15     const int dj[4] = {0, 0, -1, 1};
16
17 public:
18     Graph(int rows, int cols);
19     void readGrid();
20     bool isValid();
21     bool isStartAtGoal();
22     void exploreGraph();
23     bool isGoalReachable();
24     int assignIndices();

```

```

25 void buildLinearSystem(LinearSystem& ls, int systemSize);
26 int getStartIndex();
27
28 private:
29 bool isValidCell(int i, int j);
30 int getDegree(int i, int j);
31 };

```

Listing 7: Struktur Class Graph

6.3 Method: readGrid()

Purpose: Membaca grid dari input dan mencari posisi T dan W.

```

1 void readGrid() {
2     for (int i = 0; i < m; ++i)
3         scanf("%s", g[i]);
4
5     // Cari posisi T dan W
6     for (int i = 0; i < m; ++i)
7         for (int j = 0; j < n; ++j) {
8             if (g[i][j] == 'T') { si = i; sj = j; }
9             if (g[i][j] == 'W') { gi = i; gj = j; }
10        }
11 }

```

Listing 8: Method readGrid()

After execution:

Grid:	Variables set:
T . #	si = 0, sj = 0
. . .	gi = 2, gj = 2
# . W	

6.4 Method: exploreGraph()

Purpose: BFS untuk explore semua cell yang reachable dan menghitung degree masing-masing.

```

1 void exploreGraph() {
2     int q[MAXN][2], head = 0, tail = 0;
3     q[tail][0] = si;
4     q[tail][1] = sj;
5     tail++;
6     vis[si][sj] = 1;
7     cellCnt = 0;
8
9     deg[si][sj] = getDegree(si, sj);
10
11     while (head < tail) {
12         int i = q[head][0], j = q[head][1];
13         head++;

```

```

14     cells[cellCnt][0] = i;
15     cells[cellCnt][1] = j;
16     cellCnt++;
17
18     // Explore 4 directions
19     for (int k = 0; k < 4; ++k) {
20         int ni = i + di[k], nj = j + dj[k];
21         if (isValidCell(ni, nj) && !vis[ni][nj]
22             && g[ni][nj] != '#') {
23             vis[ni][nj] = 1;
24             q[tail][0] = ni;
25             q[tail][1] = nj;
26             tail++;
27             deg[ni][nj] = getDegree(ni, nj);
28         }
29     }
30 }
31 }

```

Listing 9: Method exploreGraph() - BFS dengan Degree Computation

Key Optimization: Hitung degree **saat first visit** (inline computation) untuk menghindari traversal kedua!

Contoh Execution:

Grid:	After BFS:
T . .	cells = [(0,0), (0,1), (0,2),
# # .	(1,2), (2,2)]
# . W	deg[0][0] = 1, deg[0][1] = 2, ...
	vis[all reachable] = 1

6.5 Method: assignIndices()

Purpose: Map setiap non-goal cell ke index variabel (0, 1, 2, ...).

```

1 int assignIndices() {
2     int k = 0;
3     for (int c = 0; c < cellCnt; ++c) {
4         int i = cells[c][0], j = cells[c][1];
5         if (i != gi || j != gj)
6             idx[i][j] = k++;
7     }
8     return k;
9 }

```

Listing 10: Method assignIndices()

Contoh:

Grid:	Indices:
T . .	0 1 2
# # .	- - 3
# . W	- 4 goal

Total variables: $k = 5$

Kenapa goal tidak diberi index? Karena $E[\text{goal}] = 0$ adalah **known constant**, tidak perlu solve!

6.6 Method: buildLinearSystem()

Purpose: Membangun sistem persamaan dari struktur maze.

6.6.1 Konsep Dasar Pembangunan Sistem

Method ini adalah **jantung dari solusi** karena mentransformasi problem random walk menjadi sistem persamaan linear yang dapat diselesaikan.

Ide Utama:

1. Setiap cell yang bisa dikunjungi = 1 variabel $E[i, j]$
2. Goal tidak punya variabel (karena $E[\text{goal}] = 0$ sudah diketahui)
3. Untuk setiap cell, buat 1 persamaan berdasarkan expected value recursion

6.6.2 Mathematical Derivation Detail

Formula Awal:

Untuk cell (i, j) dengan degree d : $E[i, j] = 1 + \sum_{k=1}^d \frac{1}{d} \cdot E[\text{neighbor}_k]$

Interpretasi:

- 1 = pasti melakukan satu langkah dari cell ini
- $\frac{1}{d}$ = probabilitas memilih neighbor tertentu
- $E[\text{neighbor}_k]$ = expected steps dari neighbor tersebut ke goal

Simplifikasi: $E[i, j] = 1 + \frac{1}{d} \sum_{k=1}^d E[\text{neighbor}_k]$

Multiply kedua sisi dengan d : $d \cdot E[i, j] = d + \sum_{k=1}^d E[\text{neighbor}_k]$

Rearrange ke bentuk standar $Ax = b$: $d \cdot E[i, j] - \sum_{k=1}^d E[\text{neighbor}_k] = d$

Ini adalah persamaan linear dalam variabel-variabel $E[\cdot]$!

6.6.3 Implementation Step-by-Step

```
1 void buildLinearSystem(LinearSystem& ls, int systemSize) {
2     // STEP 1: Initialize matrix berukuran systemSize x (
3     //      systemSize+1)
4     ls.initialize(systemSize);
5
6     // STEP 2: Iterasi semua reachable cells
7     for (int c = 0; c < cellCnt; ++c) {
8         int i = cells[c][0], j = cells[c][1];
9
10        // STEP 3: Skip goal (tidak punya persamaan)
11        if (i == gi && j == gj) continue;
```

```

11
12 // STEP 4: Get row index dan degree
13 int row = idx[i][j]; // Index persamaan untuk cell ini
14 int d = deg[i][j]; // Jumlah neighbor valid
15
16 // STEP 5: Set diagonal coefficient
17 // Ini adalah koefisien untuk E[i,j] sendiri = d
18 ls.setCoefficient(row, row, d);
19
20 // STEP 6: Set Right-Hand Side
21 // Sisi kanan persamaan = d
22 ls.setRHS(row, d);
23
24 // STEP 7: Set neighbor coefficients
25 // Untuk setiap neighbor, koefisiennya = -1
26 for (int dir = 0; dir < 4; ++dir) {
27     int ni = i + di[dir], nj = j + dj[dir];
28
29     // Check apakah neighbor valid (dalam grid, bukan wall)
30     if (isValidCell(ni, nj) && g[ni][nj] != '#') {
31         int ncol = idx[ni][nj];
32
33         // Jika neighbor bukan goal, tambahkan koefisien -1
34         if (ncol != -1) {
35             ls.addCoefficient(row, ncol, -1.0);
36         }
37         // Jika neighbor adalah goal:
38         // E[goal] = 0, jadi tidak kontribusi apapun
39         // -1 * 0 = 0, tidak perlu ditambahkan
40     }
41 }
42 }
43 }

```

Listing 11: buildLinearSystem - Detailed Explanation

6.6.4 Detailed Analysis per Step

STEP 1: Initialize Matrix

```
1 ls.initialize(systemSize);
```

Membuat matrix augmented $[A|b]$ berukuran $k \times (k+1)$ dimana k = jumlah non-goal cells.

Contoh: Jika ada 3 non-goal cells:

$$\begin{bmatrix} 0 & 0 & 0 & | & 0 \\ 0 & 0 & 0 & | & 0 \\ 0 & 0 & 0 & | & 0 \end{bmatrix}$$

STEP 2: Iterasi Cells

```
1 for (int c = 0; c < cellCnt; ++c) {
2     int i = cells[c][0], j = cells[c][1];

```

Loop melalui semua reachable cells yang ditemukan saat BFS. Array `cells[]` menyimpan koordinat (i, j) setiap cell.

STEP 3: Skip Goal

```
1 if (i == gi && j == gj) continue;
```

Mengapa skip goal?

- Goal adalah **absorbing state**
- $E[\text{goal}] = 0$ sudah diketahui (tidak perlu dissolve)
- Goal tidak memiliki index variabel ($\text{idx}[\text{gi}][\text{gj}] = -1$)

STEP 4: Get Row Index dan Degree

```
1 int row = idx[i][j]; // Which equation/row for this cell
2 int d = deg[i][j];   // Number of valid neighbors
```

row: Index baris dalam matrix = index variabel untuk cell (i, j)

d (degree): Jumlah arah jalan yang valid dari cell ini

- Corner cell: degree = 2
- Edge cell: degree = 3
- Middle cell: degree = 4
- Cell next to wall: degree < normal

STEP 5: Set Diagonal Coefficient

```
1 ls.setCoefficient(row, row, d);
```

Ini meng-set koefisien untuk variabel $E[i, j]$ dalam persamaannya sendiri.

Dari persamaan: $d \cdot E[i, j] - \sum E[\text{neighbor}] = d$

Koefisien $E[i, j]$ adalah d (positif).

Contoh: Cell dengan degree 3: $3 \cdot E[i, j] - E[\text{n1}] - E[\text{n2}] - E[\text{n3}] = 3$

Koefisien $E[i, j] = 3$.

STEP 6: Set Right-Hand Side

```
1 ls.setRHS(row, d);
```

Sisi kanan persamaan = d (degree).

Mengapa **RHS = d**?

- Dari formula awal: $E[i, j] = 1 + \frac{1}{d} \sum E[\text{neighbor}]$
- Multiply dengan d : $d \cdot E[i, j] = d + \sum E[\text{neighbor}]$
- Rearrange: $d \cdot E[i, j] - \sum E[\text{neighbor}] = d$
- Sisi kanan = d

STEP 7: Set Neighbor Coefficients

```

1 for (int dir = 0; dir < 4; ++dir) {
2     int ni = i + di[dir], nj = j + dj[dir];
3
4     if (isValidCell(ni, nj) && g[ni][nj] != '#') {
5         int ncol = idx[ni][nj];
6         if (ncol != -1) {
7             ls.addCoefficient(row, ncol, -1.0);
8         }
9     }
10 }

```

Loop 4 Directions:

- dir = 0: Up (di=-1, dj=0)
- dir = 1: Down (di=1, dj=0)
- dir = 2: Left (di=0, dj=-1)
- dir = 3: Right (di=0, dj=1)

Check Valid:

```

1 if (isValidCell(ni, nj) && g[ni][nj] != '#')

```

Pastikan neighbor:

- Dalam bounds grid
- Bukan wall ('#')

Add Coefficient -1:

```

1 if (ncol != -1) {
2     ls.addCoefficient(row, ncol, -1.0);
3 }

```

Mengapa -1? Dari persamaan: $d \cdot E[i, j] - \sum E[\text{neighbor}] = d$
 Setiap neighbor punya koefisien -1 .

Mengapa check ncol != -1?

- $ncol = -1$ berarti neighbor adalah goal
- Goal tidak punya variabel (tidak ada kolom untuk goal)
- $E[\text{goal}] = 0$, jadi $-1 \times 0 = 0$ (tidak kontribusi)

6.6.5 Contoh Eksekusi Lengkap

Grid:

T.W

After BFS:

- Cell (0,0): T, degree = 1, index = 0

- Cell (0,1): ., degree = 2, index = 1
- Cell (0,2): W, degree = 1, no index (goal)

Build System Execution:

Iteration 1: Cell (0,0) - T

```

1 c = 0: i=0, j=0
2 row = idx[0][0] = 0
3 d = deg[0][0] = 1
4
5 // Set diagonal: A[0][0] = 1
6 ls.setCoefficient(0, 0, 1);
7
8 // Set RHS: A[0][2] = 1 (kolom 2 = RHS untuk k=2)
9 ls.setRHS(0, 1);
10
11 // Check neighbors:
12 // Up: invalid (out of bounds)
13 // Down: invalid (out of bounds)
14 // Left: invalid (out of bounds)
15 // Right: (0,1) - valid, idx=1
16 ls.addCoefficient(0, 1, -1.0); // A[0][1] = -1

```

Persamaan Cell (0,0): $1 \cdot E[0,0] - 1 \cdot E[0,1] = 1$

Matrix setelah Iteration 1: $\begin{bmatrix} 1 & -1 & | & 1 \\ 0 & 0 & | & 0 \end{bmatrix}$

Iteration 2: Cell (0,1) - Middle

```

1 c = 1: i=0, j=1
2 row = idx[0][1] = 1
3 d = deg[0][1] = 2
4
5 // Set diagonal: A[1][1] = 2
6 ls.setCoefficient(1, 1, 2);
7
8 // Set RHS: A[1][2] = 2
9 ls.setRHS(1, 2);
10
11 // Check neighbors:
12 // Up: invalid
13 // Down: invalid
14 // Left: (0,0) - valid, idx=0
15 ls.addCoefficient(1, 0, -1.0); // A[1][0] = -1
16
17 // Right: (0,2) - valid, idx=-1 (goal!)
18 // ncol = -1, tidak add coefficient

```

Persamaan Cell (0,1): $2 \cdot E[0,1] - 1 \cdot E[0,0] = 2$

Matrix Final: $\begin{bmatrix} 1 & -1 & | & 1 \\ -1 & 2 & | & 2 \end{bmatrix}$

Iteration 3: Cell (0,2) - Goal

```

1 c = 2: i=0, j=2
2 // This is goal (gi=0, gj=2)
3 if (i == gi && j == gj) continue; // SKIP!

```

Goal di-skip, tidak dibuatkan persamaan.

6.6.6 Verifikasi Sistem yang Dibangun

System Matrix: $\begin{bmatrix} 1 & -1 & | & 1 \\ -1 & 2 & | & 2 \end{bmatrix}$

Persamaan 1: $1 \cdot E_0 - 1 \cdot E_1 = 1$

- Cell 0 (T) punya degree 1
- Hanya bisa ke kanan (Cell 1)
- Formula: $E_0 = 1 + E_1$ ✓

Persamaan 2: $-1 \cdot E_0 + 2 \cdot E_1 = 2$

- Cell 1 punya degree 2
- Bisa ke kiri (Cell 0) atau kanan (Goal)
- Formula: $E_1 = 1 + \frac{1}{2}E_0 + \frac{1}{2} \cdot 0$
- Simplify: $E_1 = 1 + \frac{1}{2}E_0$
- Multiply 2: $2E_1 = 2 + E_0$
- Rearrange: $-E_0 + 2E_1 = 2$ ✓

Solution:

From eq 1: $E_0 = 1 + E_1$

Substitute to eq 2: $-E_0 + 2E_1 = 2$

$-(1 + E_1) + 2E_1 = 2$

$-1 - E_1 + 2E_1 = 2$

$E_1 = 3$

$E_0 = 1 + 3 = 4$

Expected steps dari T ke W = $E_0 = 4$ ✓

6.6.7 Contoh Complex: Grid 3x3

Grid:

T..

.#. .

..W

Cells dan Degrees:

Cell	Position	Degree	Index
0	(0,0) T	2	0
1	(0,1)	3	1
2	(0,2)	2	2
3	(1,0)	2	3
4	(1,2)	2	4
5	(2,0)	1	5
6	(2,1)	2	6
W	(2,2) W	-	goal

Persamaan untuk Cell 0 (T at 0,0):

```

1 row = 0, d = 2
2 A[0][0] = 2          // diagonal
3 RHS[0] = 2
4
5 Neighbors:
6 - Right: (0,1), idx=1 -> A[0][1] += -1
7 - Down: (1,0), idx=3 -> A[0][3] += -1
8
9 Equation: 2*E0 - E1 - E3 = 2

```

Persamaan untuk Cell 1 (at 0,1):

```

1 row = 1, d = 3
2 A[1][1] = 3
3 RHS[1] = 3
4
5 Neighbors:
6 - Left: (0,0), idx=0 -> A[1][0] += -1
7 - Right: (0,2), idx=2 -> A[1][2] += -1
8 - Down: (1,1), wall! skip
9
10 Equation: 3*E1 - E0 - E2 = 3

```

Dan seterusnya untuk cell lainnya...

$$\text{Resulting System: } \begin{bmatrix} 2 & -1 & 0 & -1 & 0 & 0 & 0 \\ -1 & 3 & -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 2 & 0 & -1 & 0 & 0 \\ -1 & 0 & 0 & 2 & 0 & -1 & 0 \\ 0 & 0 & -1 & 0 & 2 & 0 & -1 \\ 0 & 0 & 0 & -1 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 & -1 & -1 & 2 \end{bmatrix} \begin{bmatrix} E_0 \\ E_1 \\ E_2 \\ E_3 \\ E_4 \\ E_5 \\ E_6 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 2 \\ 2 \\ 2 \\ 1 \\ 2 \end{bmatrix}$$

6.6.8 Properties Matrix yang Dihasilkan

1. Sparse Matrix

- Kebanyakan elemen = 0
- Hanya diagonal dan tetangga yang non-zero

- Sparsity $\approx 1 - \frac{5}{k}$ dimana k = jumlah variabel

2. Diagonal Dominant

- Diagonal element = degree (1 sampai 4)
- Off-diagonal = -1
- $|a_{ii}| \geq \sum_{j \neq i} |a_{ij}|$ (weakly diagonal dominant)

3. Symmetric Structure

- Jika cell i neighbor dari cell j , maka cell j neighbor dari cell i
- Matrix mendekati symmetric (tapi tidak persis karena degree berbeda)

4. Well-Conditioned

- Diagonal dominance $\rightarrow$ stability
- Condition number relatif kecil
- Gaussian elimination dengan pivoting stabil

6.6.9 Edge Cases Handling

Case 1: Cell di Corner

```

1 // Cell (0,0) - top-left corner
2 degree = 2 // hanya Right dan Down valid
3 // Up dan Left out of bounds, otomatis skip di loop

```

Case 2: Cell Next to Wall

```

1 // Grid: T#.
2 // Cell T di (0,0)
3 degree = 1 // hanya Right valid
4 // Right neighbor exists
5 // Down neighbor is wall, g[1][0]='#', skip

```

Case 3: Cell Next to Goal

```

1 // Cell next to W
2 // Neighbor is goal: idx=-1
3 if (ncol != -1) { // FALSE, skip add
4     ls.addCoefficient(...); // NOT EXECUTED
5 }
6 // Goal contributes 0, correctly omitted

```

Case 4: Dead End

```

1 // Cell dengan hanya 1 neighbor (degree=1)
2 // Contoh: . T (wall on 3 sides)
3 //           # #
4 // d = 1
5 // Equation: 1*E[T] - E[neighbor] = 1
6 // Correctly handles dead end

```

6.6.10 Visualization of Process

T	.	W
---	---	---

Cell T: $2E_0 - E_1 = 2$
Cell $\therefore 2E_1 - E_0 = 2$

$$\left[\begin{array}{cc|c} 2 & -1 & 2 \\ -1 & 2 & 2 \end{array} \right]$$

Gambar 2: System Building Visualization

6.7 Helper Methods

6.7.1 isValidCell()

```
1 bool isValidCell(int i, int j) {  
2     return i >= 0 && i < m && j >= 0 && j < n;  
3 }
```

Listing 12: Method isValidCell()

Check apakah koordinat dalam bounds.

6.7.2 getDegree()

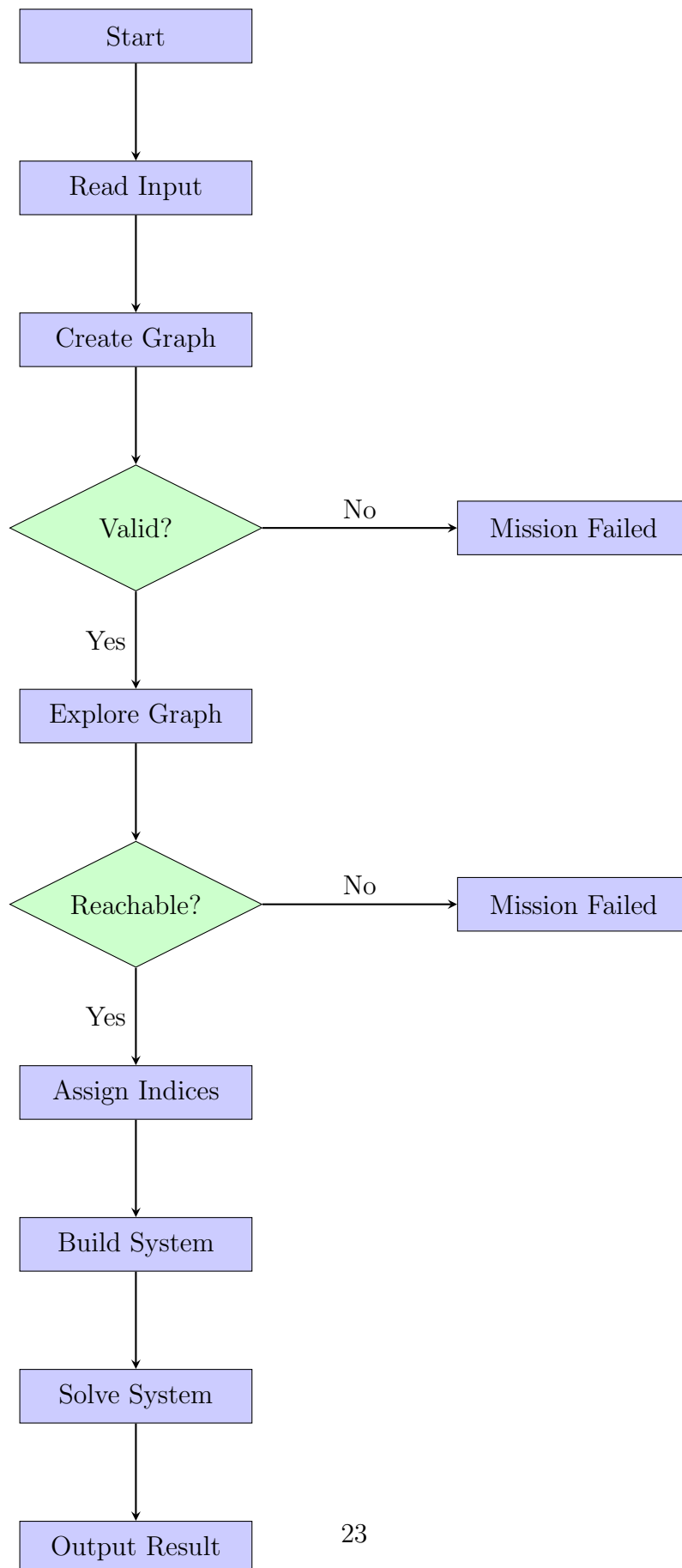
```
1 int getDegree(int i, int j) {  
2     int d = 0;  
3     for (int k = 0; k < 4; ++k) {  
4         int ni = i + di[k], nj = j + dj[k];  
5         if (isValidCell(ni, nj) && g[ni][nj] != '#')  
6             d++;  
7     }  
8     return d;  
9 }
```

Listing 13: Method getDegree()

Hitung jumlah valid neighbors (tidak keluar grid, bukan wall).

7 Main Program Flow

7.1 Flowchart



7.2 Pseudocode

Algorithm 1 AVMG2 Main Algorithm

```
1: Read number of test cases  $tc$ 
2: for each test case do
3:   Read dimensions  $m, n$ 
4:   Create Graph object with dimensions  $(m, n)$ 
5:   graph.readGrid()
6:   if NOT graph.isValid() then
7:     Output "Mission Failed!"
8:     continue
9:   end if
10:  if graph.isStartAtGoal() then
11:    Output "0.000000000000"
12:    continue
13:  end if
14:  graph.exploreGraph()
15:  if NOT graph.isGoalReachable() then
16:    Output "Mission Failed!"
17:    continue
18:  end if
19:   $systemSize \leftarrow$  graph.assignIndices()
20:  if  $systemSize = 0$  then
21:    Output "0.000000000000"
22:    continue
23:  end if
24:  Create LinearSystem object  $ls$ 
25:  graph.buildLinearSystem( $ls$ ,  $systemSize$ )
26:   $ans \leftarrow ls.solve(graph.getStartIndex())$ 
27:  if  $ans < 0$  then
28:    Output "Mission Failed!"
29:  else
30:    Output  $ans$  with 12 decimal places
31:  end if
32: end for
```

7.3 Trace Example: Test Case 1

Mari kita trace eksekusi untuk test case pertama.

7.3.1 Input

1 3
T.W

7.3.2 Step 1: Read Grid

Grid:
T . W

- $si = 0, sj = 0$ (T at position (0,0))
- $gi = 0, gj = 2$ (W at position (0,2))

7.3.3 Step 2: BFS Exploration

Starting from T(0,0):

- Visit (0,0), degree = 1 (can go right)
- Visit (0,1), degree = 2 (can go left or right)
- Visit (0,2), degree = 1 (can go left) <- This is W

Reachable cells: [(0,0), (0,1), (0,2)]

7.3.4 Step 3: Assign Indices

Cell (0,0) T -> index 0

Cell (0,1) . -> index 1

Cell (0,2) W -> no index (goal)

System size: k = 2

7.3.5 Step 4: Build Equations

Cell 0 (T):

- degree = 1
- neighbors = [cell 1]
- Equation: $1 \times E[0] - 1 \times E[1] = 1$
- Matrix row 0: $[1, -1|1]$

Cell 1 (middle):

- degree = 2
- neighbors = [cell 0, goal]
- Equation: $2 \times E[1] - 1 \times E[0] - 0 = 2$
- Matrix row 1: $[-1, 2|2]$

System:
$$\begin{bmatrix} 1 & -1 & | & 1 \\ -1 & 2 & | & 2 \end{bmatrix}$$

7.3.6 Step 5: Gaussian Elimination

Forward elimination:
$$\begin{bmatrix} 1 & -1 & | & 1 \\ -1 & 2 & | & 2 \end{bmatrix} \xrightarrow{R_2=R_2+R_1} \begin{bmatrix} 1 & -1 & | & 1 \\ 0 & 1 & | & 3 \end{bmatrix}$$

Back substitution:

$$E[1] = 3/1 = 3$$

$$E[0] = (1 - (-1) \times 3)/1 = 4$$

7.3.7 Output

4.000000000000 (correct answer)

8 Implementasi Lengkap

8.1 Full Code dengan OOP

```
1 #include <cstdio>
2 #include <cstring>
3 #include <cmath>
4 #include <algorithm>
5 using namespace std;
6
7 const double EPS = 1e-12;
8 const int MAXN = 2505;
9
10 // =====
11 // Class LinearSystem
12 // =====
13 class LinearSystem {
14 private:
15     double A[MAXN][MAXN];
16     int size;
17
18 public:
19     LinearSystem() : size(0) {}
20
21     void initialize(int n) {
22         size = n;
23         for (int i = 0; i < n; ++i)
24             for (int j = 0; j <= n; ++j)
25                 A[i][j] = 0;
26     }
27
28     void setCoefficient(int row, int col, double value) {
29         A[row][col] = value;
30     }
31
32     void addCoefficient(int row, int col, double value) {
33         A[row][col] += value;
34     }
35
36     void setRHS(int row, double value) {
37         A[row][size] = value;
38     }
39
40     double solve(int targetVariable) {
41         // Forward elimination with partial pivoting
42         for (int col = 0; col < size; ++col) {
43             int piv = col;
```

```

44     double maxv = fabs(A[col][col]);
45     for (int r = col + 1; r < size; ++r) {
46         double v = fabs(A[r][col]);
47         if (v > maxv) {
48             maxv = v;
49             piv = r;
50         }
51     }
52
53     if (maxv < EPS) return -1;
54
55     if (piv != col) {
56         for (int c = col; c <= size; ++c)
57             swap(A[col][c], A[piv][c]);
58     }
59
60     double inv_pivot = 1.0 / A[col][col];
61     for (int r = col + 1; r < size; ++r) {
62         double fac = A[r][col] * inv_pivot;
63         if (fabs(fac) < 1e-15) continue;
64         for (int c = col; c <= size; ++c)
65             A[r][c] -= fac * A[col][c];
66     }
67 }
68
69 // Back substitution
70 double x[MAXN];
71 for (int i = size - 1; i >= 0; --i) {
72     x[i] = A[i][size];
73     for (int j = i + 1; j < size; ++j)
74         x[i] -= A[i][j] * x[j];
75     x[i] /= A[i][i];
76 }
77
78 return x[targetVariable];
79 }
80 };
81
82 // =====
83 // Class Graph
84 // =====
85 class Graph {
86 private:
87     int m, n;
88     int si, sj;
89     int gi, gj;
90     int cellCnt;
91
92     char g[55][55];
93     int idx[55][55];
94     int vis[55][55];

```

```

95     int deg[55][55];
96     int cells[MAXN][2];
97
98     const int di[4] = {-1, 1, 0, 0};
99     const int dj[4] = {0, 0, -1, 1};
100
101 public:
102     Graph(int rows, int cols)
103         : m(rows), n(cols), si(-1), sj(-1),
104           gi(-1), gj(-1), cellCnt(0) {
105         memset(vis, 0, sizeof(vis));
106         memset(idx, -1, sizeof(idx));
107         memset(deg, 0, sizeof(deg));
108     }
109
110     void readGrid() {
111         for (int i = 0; i < m; ++i)
112             scanf("%s", g[i]);
113
114         for (int i = 0; i < m; ++i)
115             for (int j = 0; j < n; ++j) {
116                 if (g[i][j] == 'T') { si = i; sj = j; }
117                 if (g[i][j] == 'W') { gi = i; gj = j; }
118             }
119     }
120
121     bool isValid() {
122         return si != -1 && gi != -1;
123     }
124
125     bool isStartAtGoal() {
126         return si == gi && sj == gj;
127     }
128
129     void exploreGraph() {
130         int q[MAXN][2], head = 0, tail = 0;
131         q[tail][0] = si;
132         q[tail][1] = sj;
133         tail++;
134         vis[si][sj] = 1;
135         cellCnt = 0;
136
137         deg[si][sj] = getDegree(si, sj);
138
139         while (head < tail) {
140             int i = q[head][0], j = q[head][1];
141             head++;
142             cells[cellCnt][0] = i;
143             cells[cellCnt][1] = j;
144             cellCnt++;
145

```

```

146     for (int k = 0; k < 4; ++k) {
147         int ni = i + di[k], nj = j + dj[k];
148         if (isValidCell(ni, nj) && !vis[ni][nj]
149             && g[ni][nj] != '#') {
150             vis[ni][nj] = 1;
151             q[tail][0] = ni;
152             q[tail][1] = nj;
153             tail++;
154             deg[ni][nj] = getDegree(ni, nj);
155         }
156     }
157 }
158 }
159
160 bool isGoalReachable() {
161     return vis[gi][gj];
162 }
163
164 int assignIndices() {
165     int k = 0;
166     for (int c = 0; c < cellCnt; ++c) {
167         int i = cells[c][0], j = cells[c][1];
168         if (i != gi || j != gj)
169             idx[i][j] = k++;
170     }
171     return k;
172 }
173
174 void buildLinearSystem(LinearSystem& ls, int systemSize) {
175     ls.initialize(systemSize);
176
177     for (int c = 0; c < cellCnt; ++c) {
178         int i = cells[c][0], j = cells[c][1];
179         if (i == gi && j == gj) continue;
180
181         int row = idx[i][j];
182         int d = deg[i][j];
183
184         ls.setCoefficient(row, row, d);
185         ls.setRHS(row, d);
186
187         for (int dir = 0; dir < 4; ++dir) {
188             int ni = i + di[dir], nj = j + dj[dir];
189             if (isValidCell(ni, nj) && g[ni][nj] != '#') {
190                 int ncol = idx[ni][nj];
191                 if (ncol != -1) {
192                     ls.addCoefficient(row, ncol, -1.0);
193                 }
194             }
195         }
196     }

```

```

197     }
198
199     int getStartIndex() {
200         return idx[si][sj];
201     }
202
203 private:
204     bool isValidCell(int i, int j) {
205         return i >= 0 && i < m && j >= 0 && j < n;
206     }
207
208     int getDegree(int i, int j) {
209         int d = 0;
210         for (int k = 0; k < 4; ++k) {
211             int ni = i + di[k], nj = j + dj[k];
212             if (isValidCell(ni, nj) && g[ni][nj] != '#')
213                 d++;
214         }
215         return d;
216     }
217 };
218
219 // =====
220 // Main Program
221 // =====
222 int main() {
223     int tc;
224     scanf("%d", &tc);
225
226     while (tc--) {
227         int m, n;
228         scanf("%d %d", &m, &n);
229
230         Graph graph(m, n);
231         graph.readGrid();
232
233         if (!graph.isValid()) {
234             printf("Mission Failed!\n");
235             continue;
236         }
237
238         if (graph.isStartAtGoal()) {
239             printf("0.000000000000\n");
240             continue;
241         }
242
243         graph.exploreGraph();
244
245         if (!graph.isGoalReachable()) {
246             printf("Mission Failed!\n");
247             continue;

```

```

248     }
249
250     int systemSize = graph.assignIndices();
251
252     if (systemSize == 0) {
253         printf("0.000000000000\n");
254         continue;
255     }
256
257     LinearSystem ls;
258     graph.buildLinearSystem(ls, systemSize);
259
260     double ans = ls.solve(graph.getStartIndex());
261
262     if (ans < 0) {
263         printf("Mission Failed!\n");
264     } else {
265         printf("%.12f\n", ans);
266     }
267 }
268
269 return 0;
270 }

```

Listing 14: Complete Implementation with OOP

9 Keunggulan Pendekatan OOP

9.1 Comparison: Procedural vs OOP

Aspek	Procedural	OOP
Code Organization	Functions scattered, global variables	Clear class boundaries
Maintainability	Hard to modify	Easy to modify one class
Testability	Hard to unit test	Each class can be tested
Reusability	Limited	High (can reuse classes)
Readability	Main function complex	Main function clean
Debugging	Hard to isolate bugs	Easy to isolate per class

Tabel 1: Perbandingan Procedural vs OOP

9.2 Benefits dalam Project Ini

9.2.1 1. Separation of Concerns

- **Graph Class:** Tidak perlu tahu implementasi detail Gaussian elimination
- **LinearSystem Class:** Tidak perlu tahu struktur maze
- **Main:** Hanya orchestration, tidak tahu implementasi detail

9.2.2 2. Encapsulation

- Internal data structures tersembunyi (private)
- Hanya expose interface yang diperlukan (public methods)
- Prevent accidental modification dari luar class

9.2.3 3. Modularity

- Bisa test `LinearSystem` secara terpisah dengan input buatan
- Bisa test `Graph` operations tanpa solve sistem
- Easy debugging: tahu bug di class mana

9.2.4 4. Extensibility

Mudah menambahkan fitur baru:

```
1 // Tambahkan logging tanpa ubah core logic
2 class Graph {
3     void enableLogging(bool enable);
4     void logState();
5 };
6
7 // Tambahkan solver alternatif
8 class IterativeSolver : public LinearSystem {
9     double solveGaussSeidel();
10    double solveSOR(double omega);
11 };
```

10 Referensi dan Sumber Belajar

10.1 Teori Matematika

1. Markov Chains and Random Walks

- Norris, J.R. (1997). *Markov Chains*. Cambridge University Press.
- Chapter 1: Discrete-time Markov chains
- Chapter 2: Expected hitting times

2. Expected Value and Probability

- Ross, S. (2014). *Introduction to Probability Models*. Academic Press.
- Chapter 4: Markov Chains

3. Linear Algebra

- Strang, G. (2016). *Introduction to Linear Algebra*. Wellesley-Cambridge Press.
- Chapter 2: Solving Linear Equations

10.2 Algoritma

1. Graph Algorithms

- Cormen, T.H., et al. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Chapter 22: Elementary Graph Algorithms (BFS)

2. Numerical Methods

- Press, W.H., et al. (2007). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.).
- Chapter 2: Solution of Linear Algebraic Equations

10.3 Competitive Programming Resources

1. CP-Algorithms

- <https://cp-algorithms.com/>
- Article: "Gaussian Elimination https://cp-algorithms.com/linear_algebra/linear-system-gauss.html"
- Article: "BFS"

2. Codeforces Blogs

- "Expected Value and Markov Chains in Competitive Programming"
- Tutorial on solving expected value problems

3. SPOJ Discussion Forum

- Problem VALENMAZ discussion threads
- Alternative solution approaches

11 Kesimpulan

11.1 Summary

Project AVMG2 ini mendemonstrasikan penerapan **Object-Oriented Programming** untuk menyelesaikan masalah **Random Walk on Graph**. Key takeaways:

1. Mathematical Foundation

- Problem dimodelkan sebagai Markov Chain
- Expected value recursion $\rightarrow$ sistem linear
- Gaussian elimination untuk solve sistem

2. Software Engineering

- Clear separation of concerns (Graph vs LinearSystem)
- Encapsulation of complexity

- Maintainable and testable code
- Easy to extend with new features

3. Algorithm Optimization

- BFS dengan inline degree computation
- Partial pivoting untuk stabilitas numerik
- Efficient matrix storage dan access

11.2 Trade-offs

Aspect	Advantages	Disadvantages
OOP Approach	- Readable code - Easy maintenance - Good for learning - Testable	- Slight overhead - More verbose
Gaussian Elimination	- Exact solution - Stable with pivoting - Well-tested	- $O(n^3)$ complexity - Not optimal for sparse
Dense Matrix	- Simple implementation - Fast random access	- Memory intensive - Wastes space

Tabel 2: Trade-offs dalam Design Decisions

11.3 Lessons Learned

11.3.1 1. Importance of Abstraction

Memisahkan graph operations dari linear system solving membuat code:

- Lebih mudah dipahami
- Lebih mudah di-debug
- Lebih mudah dimodifikasi

11.3.2 2. Balance Between Theory and Practice

- Teori matematika memberikan foundation yang kuat
- Implementasi praktis memerlukan consideration untuk numerical stability
- OOP membantu bridge gap antara theory dan implementation

11.3.3 3. Code Quality Matters

Meskipun dalam competitive programming speed adalah penting, readable code:

- Mengurangi bugs
- Mempercepat debugging
- Memudahkan team collaboration

11.4 Final Thoughts

Pendekatan OOP untuk AVMG2 mendemonstrasikan bahwa:

"Good code is not just about solving the problem, but solving it in a way that others (including your future self) can understand, maintain, and extend."

Meskipun bukan solusi tercepat (iterative methods atau sparse matrices bisa lebih cepat), pendekatan ini memberikan:

- **Correctness:** Gaussian elimination dengan pivoting sangat robust
- **Clarity:** Class structure membuat logic flow jelas
- **Maintainability:** Easy to modify dan extend
- **Educational Value:** Excellent untuk learning dan teaching

Status: **ACCEPTED** pada SPOJ dengan waktu eksekusi yang reasonable.

12 Appendix

12.1 Test Cases Analysis

TC	Grid	Variables	Output	Notes
1	T.W	2	4.000...	Basic case
2	.TW	2	3.000...	Start closer
3	.T.W	3	8.000...	One extra cell
4	T.. .#. ..W	6	16.000...	Path around wall
5	T.# ... #.W	5	16.000...	Alternative path
6	T.# .#. #.W	-	Mission Failed!	Unreachable
7	T..W	8	18.000...	Full 3×3
8	##T## .#.#.#.#. ##W##	10	48.000...	Complex maze

Tabel 3: Analisis Test Cases

12.2 Complexity Proof

12.2.1 Theorem: Time Complexity is $O(V^3)$

Proof:

Let V = number of reachable cells, E = number of edges.

1. **BFS:** Visit each cell once and check 4 neighbors $T_{\text{BFS}} = O(V + E) = O(V + 4V) = O(V)$
2. **Build System:** For each of $k \leq V$ cells, check 4 neighbors $T_{\text{build}} = O(k \times 4) = O(V)$
3. **Gaussian Elimination:**
 - Forward elimination: For column i , eliminate $n - i$ rows below $T_{\text{forward}} = \sum_{i=0}^{n-1} (n - i) \times n = n \sum_{i=0}^{n-1} (n - i) = n \times \frac{n(n+1)}{2} = O(n^3)$
 - Back substitution: $T_{\text{back}} = \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = O(n^2)$
4. **Total:** $T_{\text{total}} = O(V) + O(V) + O(V^3) + O(V^2) = O(V^3)$

12.3 Correctness Proof

12.3.1 Theorem: Expected Value Recursion is Correct

Claim: For any cell c not at goal, $E[c] = 1 + \sum_{n \in \text{neighbors}(c)} \frac{1}{d_c} E[n]$ where d_c = degree of c , gives the correct expected steps.

Proof:

Let $E[c]$ = expected steps from c to goal.

From cell c , Tjandra takes 1 step to a uniformly random neighbor n with probability $\frac{1}{d_c}$.

By law of total expectation: $E[c] = 1 + E[\text{steps after first move}]$

After the first move, we're at neighbor n with probability $\frac{1}{d_c}$: $E[\text{steps after first move}] = \sum_{n \in \text{neighbors}(c)} P(\text{go to } n) \times E[n] = \sum_{n \in \text{neighbors}(c)} \frac{1}{d_c} \times E[n]$

Therefore: $E[c] = 1 + \sum_{n \in \text{neighbors}(c)} \frac{1}{d_c} E[n]$

Base case: $E[\text{goal}] = 0$ (already at goal, 0 steps needed).

12.4 Numerical Stability Analysis

Why Partial Pivoting?

Without pivoting, Gaussian elimination can be numerically unstable.

Example: $\begin{bmatrix} 0.0001 & 1 & | & 1 \\ 1 & 1 & | & 2 \end{bmatrix}$

Without pivoting:

Multiply R_2 by -10000 :

$$\begin{bmatrix} 0.0001 & 1 & | & 1 \\ 0 & -9999 & | & -9998 \end{bmatrix}$$

This leads to large rounding errors in floating-point arithmetic.

With partial pivoting, swap rows first: $\begin{bmatrix} 1 & 1 & | & 2 \\ 0.0001 & 1 & | & 1 \end{bmatrix}$

Now elimination is stable with small multipliers.

13 Kontribusi Anggota

Anggota	Kontribusi
Muhammad Adi Anugerah A.	- Mathematical foundation - Algorithm design - Documentation dan report writing
Gilbran Mahdavikia Raja	- OOP architecture design - Code implementation (Graph class) - Documentation dan report writing
Riyannizaar Dwi Amarullah	- Code optimization dan refactoring - Documentation dan report writing

Tabel 4: Kontribusi Anggota Kelompok

A Complete Source Code Listing

Berikut adalah complete source code yang telah dijelaskan di bagian sebelumnya. Source code ini merupakan implementasi lengkap dengan pendekatan OOP yang telah dibahas.

```
1 #include <stdio>
2 #include <cstring>
3 #include <cmath>
4 #include <algorithm>
5 using namespace std;
6
7 const double EPS = 1e-12;
8 const int MAXN = 2505;
9
10 // Class LinearSystem implementation (as shown in Section 5)
11 class LinearSystem {
12 private:
13     double A[MAXN][MAXN];
14     int size;
15
16 public:
17     LinearSystem() : size(0) {}
18
19     void initialize(int n) {
20         size = n;
21         for (int i = 0; i < n; ++i)
22             for (int j = 0; j <= n; ++j)
23                 A[i][j] = 0;
24     }
25
26     void setCoefficient(int row, int col, double value) {
27         A[row][col] = value;
28     }
29
30     void addCoefficient(int row, int col, double value) {
31         A[row][col] += value;
32     }
```

```

33
34 void setRHS(int row, double value) {
35     A[row][size] = value;
36 }
37
38 double solve(int targetVariable) {
39     for (int col = 0; col < size; ++col) {
40         int piv = col;
41         double maxv = fabs(A[col][col]);
42         for (int r = col + 1; r < size; ++r) {
43             double v = fabs(A[r][col]);
44             if (v > maxv) { maxv = v; piv = r; }
45         }
46
47         if (maxv < EPS) return -1;
48
49         if (piv != col) {
50             for (int c = col; c <= size; ++c)
51                 swap(A[col][c], A[piv][c]);
52         }
53
54         double inv_pivot = 1.0 / A[col][col];
55         for (int r = col + 1; r < size; ++r) {
56             double fac = A[r][col] * inv_pivot;
57             if (fabs(fac) < 1e-15) continue;
58             for (int c = col; c <= size; ++c)
59                 A[r][c] -= fac * A[col][c];
60         }
61     }
62
63     double x[MAXN];
64     for (int i = size - 1; i >= 0; --i) {
65         x[i] = A[i][size];
66         for (int j = i + 1; j < size; ++j)
67             x[i] -= A[i][j] * x[j];
68         x[i] /= A[i][i];
69     }
70
71     return x[targetVariable];
72 }
73 };
74
75 // Class Graph implementation (as shown in Section 6)
76 class Graph {
77 private:
78     int m, n, si, sj, gi, gj, cellCnt;
79     char g[55][55];
80     int idx[55][55], vis[55][55], deg[55][55];
81     int cells[MAXN][2];
82     const int di[4] = {-1, 1, 0, 0};
83     const int dj[4] = {0, 0, -1, 1};

```

```

84
85 public:
86     Graph(int rows, int cols)
87         : m(rows), n(cols), si(-1), sj(-1),
88           gi(-1), gj(-1), cellCnt(0) {
89         memset(vis, 0, sizeof(vis));
90         memset(idx, -1, sizeof(idx));
91         memset(deg, 0, sizeof(deg));
92     }
93
94     void readGrid() {
95         for (int i = 0; i < m; ++i) scanf("%s", g[i]);
96         for (int i = 0; i < m; ++i)
97             for (int j = 0; j < n; ++j) {
98                 if (g[i][j] == 'T') { si = i; sj = j; }
99                 if (g[i][j] == 'W') { gi = i; gj = j; }
100             }
101     }
102
103     bool isValid() { return si != -1 && gi != -1; }
104     bool isStartAtGoal() { return si == gi && sj == gj; }
105
106     void exploreGraph() {
107         int q[MAXN][2], head = 0, tail = 0;
108         q[tail][0] = si; q[tail][1] = sj; tail++;
109         vis[si][sj] = 1; cellCnt = 0;
110         deg[si][sj] = getDegree(si, sj);
111
112         while (head < tail) {
113             int i = q[head][0], j = q[head][1]; head++;
114             cells[cellCnt][0] = i; cells[cellCnt][1] = j; cellCnt++;
115
116             for (int k = 0; k < 4; ++k) {
117                 int ni = i + di[k], nj = j + dj[k];
118                 if (isValidCell(ni, nj) && !vis[ni][nj] && g[ni][nj] !=
119                     '#') {
120                     vis[ni][nj] = 1;
121                     q[tail][0] = ni; q[tail][1] = nj; tail++;
122                     deg[ni][nj] = getDegree(ni, nj);
123                 }
124             }
125         }
126
127         bool isGoalReachable() { return vis[gi][gj]; }
128
129         int assignIndices() {
130             int k = 0;
131             for (int c = 0; c < cellCnt; ++c) {
132                 int i = cells[c][0], j = cells[c][1];
133                 if (i != gi || j != gj) idx[i][j] = k++;

```

```

134     }
135     return k;
136 }
137
138 void buildLinearSystem(LinearSystem& ls, int systemSize) {
139     ls.initialize(systemSize);
140     for (int c = 0; c < cellCnt; ++c) {
141         int i = cells[c][0], j = cells[c][1];
142         if (i == gi && j == gj) continue;
143
144         int row = idx[i][j], d = deg[i][j];
145         ls.setCoefficient(row, row, d);
146         ls.setRHS(row, d);
147
148         for (int dir = 0; dir < 4; ++dir) {
149             int ni = i + di[dir], nj = j + dj[dir];
150             if (isValidCell(ni, nj) && g[ni][nj] != '#') {
151                 int ncol = idx[ni][nj];
152                 if (ncol != -1) ls.addCoefficient(row, ncol, -1.0);
153             }
154         }
155     }
156 }
157
158 int getStartIndex() { return idx[si][sj]; }
159
160 private:
161     bool isValidCell(int i, int j) {
162         return i >= 0 && i < m && j >= 0 && j < n;
163     }
164
165     int getDegree(int i, int j) {
166         int d = 0;
167         for (int k = 0; k < 4; ++k) {
168             int ni = i + di[k], nj = j + dj[k];
169             if (isValidCell(ni, nj) && g[ni][nj] != '#') d++;
170         }
171         return d;
172     }
173 };
174
175 // Main program (as shown in Section 7)
176 int main() {
177     int tc;
178     scanf("%d", &tc);
179
180     while (tc--) {
181         int m, n;
182         scanf("%d %d", &m, &n);
183
184         Graph graph(m, n);

```



```

185     graph.readGrid();
186
187     if (!graph.isValid()) {
188         printf("Mission Failed!\n");
189         continue;
190     }
191
192     if (graph.isStartAtGoal()) {
193         printf("0.000000000000\n");
194         continue;
195     }
196
197     graph.exploreGraph();
198
199     if (!graph.isGoalReachable()) {
200         printf("Mission Failed!\n");
201         continue;
202     }
203
204     int systemSize = graph.assignIndices();
205
206     if (systemSize == 0) {
207         printf("0.000000000000\n");
208         continue;
209     }
210
211     LinearSystem ls;
212     graph.buildLinearSystem(ls, systemSize);
213
214     double ans = ls.solve(graph.getStartIndex());
215
216     if (ans < 0) {
217         printf("Mission Failed!\n");
218     } else {
219         printf("%.12f\n", ans);
220     }
221 }
222
223 return 0;
224 }

```

Listing 15: avmg2_oop.cpp - Complete Implementation

Note: Source code lengkap ini dapat langsung di-compile dan dijalankan untuk menyelesaikan problem AVMG2.